Chapter 1.1: Introduction to Databases

1. Core Definitions

- **Data:** Known facts that can be recorded and have implicit meaning.
  - *Note:* In database literature, the word "data" is used as both singular and plural.
  - *Example:* Names, phone numbers, and addresses stored in a mobile phone or an Excel sheet.
- **Database:** A collection of related data.
  - *Broad Definition:* Any collection of data (e.g., the words on this page).
  - *Technical Definition:* A collection of data that represents a real-world aspect, is logically coherent, and is designed for a specific purpose.
- **Miniworld (or Universe of Discourse - UoD):** The specific part of the real world that a database represents.
  - *Example:* If a database tracks a store's inventory, the "miniworld" is that store's stock and sales, not the whole world.
- **Database Management System (DBMS):** A computerized software system that enables users to create and maintain a database. It is general-purpose software.
- **Database System:** The combination of the **Database** and the **DBMS software** working together.
- **Meta-data:** The "database definition" or descriptive information stored by the DBMS (such as data types, structures, and constraints).

---

2. The Three Implicit Properties of a Database

A random pile of data is not a database. To be considered a database, it must have:

1. **Representation of Reality:** It represents a "miniworld." When the real world changes (e.g., a customer buys a camera), the database must be updated to reflect this change.
2. **Logical Coherence:** The data is not random; it has inherent meaning and logic connecting it.
3. **Specific Purpose:** It is designed for a specific group of users and specific applications.

---

3. Scale and Complexity

Databases can range from very simple to massive systems:

- **Small:** A personal address book with a few hundred records.
- **Large:** A library catalog with 500,000+ entries organized by author, title, and subject.
- **Massive:**
  - **Facebook:** Manages billions of users, friend connections, privacy settings, and posts.
  - **Amazon:** Has 60+ million active users and millions of products. It occupies over **42 terabytes** of storage (10^12 bytes) across hundreds of servers.

---

4. Manual vs. Computerized Databases

- **Manual:** Created and maintained by hand (e.g., a physical library card catalog).
- **Computerized:** Maintained by software. This can be done via:
  - *Custom programs:* Written specifically for one task (e.g., early airline reservation systems).
  - *DBMS:* General-purpose software (the focus of this text).

---

5. Key Functions of a DBMS

The DBMS facilitates several critical processes:

- **Defining:** Specifying what type of data will be stored (structure, data types, constraints). This is stored as **meta-data**.

- **Constructing:** The actual process of storing the data on a storage medium (hard drive, server, etc.).
- **Manipulating:**
  - **Querying:** Retrieving specific data (e.g., "Show me all books by this author").
  - **Updating:** Changing data to match real-world events.
  - **Reporting:** Generating summaries from the data.
- **Sharing:** Allowing multiple users or programs to access the database at the same time.
- **Protecting:**
  - *System Protection:* Prevents data loss from crashes or hardware failure.
  - *Security Protection:* Prevents unauthorized or malicious access.
- **Maintaining:** Allows the system to evolve over many years as requirements change.

Chapter 1.2: A Database Example & The System Environment

1. The "University" Database Example

To understand how a database works, the text uses a simplified example of a **UNIVERSITY** database that maintains information about students, courses, and grades.

A. Database Structure (Files and Records)

The database is organized into **five files** (conceptually, a collection of records). Each file contains records of the same type.

1. **STUDENT File:** Stores information on each student.
   - *Fields:* Name, Student_number, Class (e.g., '1' for freshman), Major (e.g., 'CS').
2. **COURSE File:** Stores data on specific courses.
   - *Fields:* Course_name, Course_number, Credit_hours, Department.
3. **SECTION File:** Stores data on specific class sections (instances of a course).
   - *Fields:* Section_identifier, Course_number, Semester, Year, Instructor.
4. **GRADE_REPORT File:** Connects students to sections and records their performance.
   - *Fields:* Student_number, Section_identifier, Grade.
5. **PREREQUISITE File:** Tracks which courses are required for others.
   - *Fields:* Course_number, Prerequisite_number.

B. Defining the Data

To properly create this database, specific details must be defined:

- **Data Elements:** The specific pieces of data in a record (e.g., a student's `Name` or `Student_number`).
- **Data Types:** The format of the data.
  - *Example:* `Name` is a string of alphabetic characters; `Student_number` is an integer; `Grade` is a single character from the set {A, B, C, D, F, I}.
- **Coding Schemes:** Using codes to represent values efficiently.
  - *Example:* Storing `1` for freshman, `2` for sophomore, etc., instead of writing out the full word.

C. Relationships Between Records

Records in different files are linked together to provide complete information.

- *Example:* The **STUDENT** file (Smith) is related to the **GRADE_REPORT** file using the `Student_number`. This allows the system to see that Smith got a 'B' in section 112.

---

2. Manipulating the Database

Manipulation involves two main activities: **Querying** and **Updating**.

- **Queries (Retrieving Data):**
  - "Retrieve the transcript of student Smith."
  - "List students who took the 'Database' course in Fall 2008."
  - "List prerequisites for the 'Database' course."

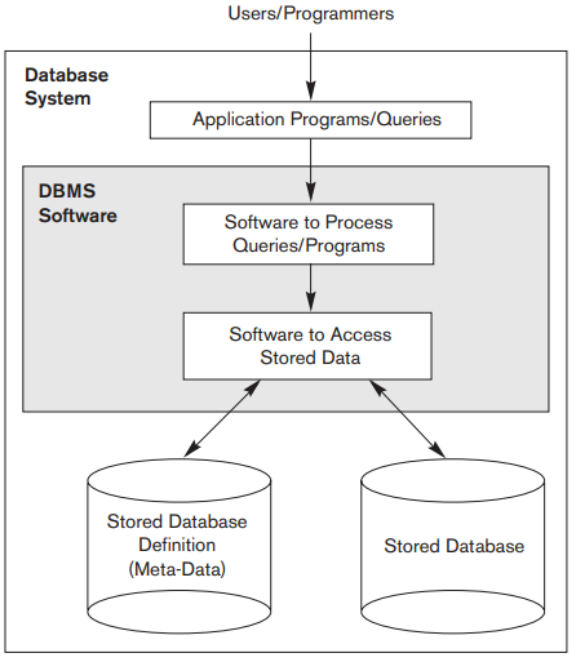
- **Updates (Changing Data):**
  - "Change Smith's class to sophomore."
  - "Create a new section for a course."
  - "Enter a grade of 'A' for Smith."

Note: These informal requests must be written precisely in a **Query Language** for the DBMS to understand them.

---

3. The Database System Environment

The database exists within a larger **Information System**. Figure 1.1 illustrates how users interact with this system.



- **Users/Programmers:** Interact with the system at the top level.
- **Application Programs/Queries:** Users send requests through these programs.
- **DBMS Software:** This central software processes the queries and accesses the stored data.
- **Stored Data:** The bottom layer consists of:
  1. **Stored Database:** The actual data (students, grades, etc.).
  2. **Stored Database Definition (Meta-data):** The description of the database structure (types, constraints).

---

4. Database Design and Life Cycle

Creating a database involves several phases managed by the IT department:

1. **Requirements Specification:** Documenting exactly what the database needs to do.
2. **Conceptual Design:** Creating a high-level representation of the structure (often using the **Entity-Relationship model**). This is easier to understand and modify than code.
3. **Logical Design:** Translating the conceptual design into a specific data model supported by a commercial DBMS (e.g., a relational model).
4. **Physical Design:** Specifying exactly how data is stored and accessed on the hardware.
5. **Implementation & Maintenance:** Populating the database with data and keeping it updated as the "miniworld" changes.

Chapter 1.3: Characteristics of the Database Approach

This section highlights the fundamental differences between the modern database approach and older "file processing" methods.

1. Traditional File Processing vs. Database Approach

- **Traditional File Processing:**
  - Each user defines and manages their own files for specific applications.
  - **Example:** A university grade office has a student file for grades, while the accounting office has a separate student file for fees.
  - **Problem:** This leads to **redundancy** (wasted space) and inconsistent data (updates must be done in two places).
- **Database Approach:**
  - A **single repository** maintains data that is defined once.
  - Many users access this single source through queries and transactions.

The database approach has **four main characteristics**:

1.3.1 Self-Describing Nature of a Database System

- **Definition:** A database system stores not just the data, but also the **complete definition and description** of the database structure and constraints.
- **Catalog (Meta-data):**
  - This definition is stored in the **DBMS catalog**.
  - The stored information is called **meta-data** (data about data).
  - It describes the structure of files, data types, and constraints.
- **General-Purpose Software:**
  - Because the structure is stored in the catalog, DBMS software is **general-purpose**. It doesn't need to be hard-coded for a specific university or bank; it just reads the catalog to understand the database structure.
  - *Contrast:* In file processing, the data definition is part of the application code (e.g., C++ struct definitions), making the program rigid and specific to one database.
- **NoSQL Exception:** Some newer NoSQL systems do not use a separate catalog but store data as "self-describing" packages (data + definition together).

1.3.2 Insulation between Programs and Data (Data Abstraction)

- **Program-Data Independence:**
  - In a DBMS, the structure of the data files is stored **separately** from the programs that access them (in the catalog).
  - *Benefit:* If you change the data structure (e.g., adding a `Birth_date` field to a STUDENT record), you only update the catalog. You do **not** need to rewrite the application programs.
  - *Contrast:* In file processing, changing a file structure often requires rewriting every program that uses that file.
- **Data Abstraction:**
  - The DBMS provides a **conceptual representation** (or Data Model) to the user, hiding the complex physical storage details (like byte length or starting positions).
  - *Example:* A user asks for "Name" and gets "Smith," without needing to know that "Name" starts at byte 1 and is 30 bytes long.
- **Program-Operation Independence:**
  - Found in object-oriented databases. Users can define operations (methods) on data. The interface (what the operation does) is separate from the implementation (how it does it), allowing the code to change without affecting the user.

1.3.3 Support of Multiple Views of the Data

- **Concept:** Different users need different perspectives (views) of the same database.

- **View Definition:** A view may contain a subset of the data or **virtual data** (derived data not explicitly stored).
- **Example:**
  - **User A (Transcript View):** Needs to see a list of courses and grades for a student.
  - **User B (Prerequisite View):** Needs to see which courses are required for others.

1.3.4 Sharing of Data and Multiuser Transaction Processing

- **Concurrency:** A multiuser DBMS allows many users to access the same data at the same time.
- **Concurrency Control Software:** Ensures that if two users try to update the same record simultaneously, they do so in a controlled way to prevent errors.
  - *Example:* Two travel agents cannot book the exact same airline seat at the same time.
- **Online Transaction Processing (OLTP):** Applications that handle many concurrent users making updates.
- **Transaction Properties:**
  - **Isolation:** Each transaction executes as if it is the only one running, even if hundreds are running in parallel.
  - **Atomicity:** Ensures a transaction is "all or nothing"—either all steps happen, or none do.

Chapter 1.4: Actors on the Scene

In small personal databases (like an address book), one person usually handles everything. However, in large organizations, many people interact with the database daily. These people are called **"Actors on the Scene."**
(Note: People who maintain the system environment but don't care about the specific data content are called "Workers Behind the Scene," covered in the next section.)

1. Database Administrators (DBA)

In any organization, shared resources need a chief manager. The **DBA** is responsible for managing the database (primary resource) and the DBMS software (secondary resource).

- **Key Responsibilities:**
  - **Authorizing Access:** Deciding who can use the database.
  - **Coordination:** Monitoring how the database is used to ensure efficiency.
  - **Resource Management:** Acquiring necessary software and hardware.
  - **Accountability:** Responsible for security breaches or poor system response times.

2. Database Designers

These actors work mostly *before* the database is built and populated. They identify what data needs to be stored and how to structure it.

- **Key Responsibilities:**
  - **Requirements Gathering:** Communicating with prospective users to understand what they need.
  - **View Development:** Creating specific "views" for different user groups that meet their specific needs.
  - **Integration:** Combining all these distinct views into one final database design that supports everyone.

3. End Users

These are the people whose jobs require access to the database to query, update, and generate reports. The database exists primarily for them. They are categorized into four types:

A. Casual End Users

- **Who they are:** Typically middle- or high-level managers.

- **Usage:** They access the database occasionally, needing different information each time.
- **Method:** They use sophisticated query interfaces to browse data.

B. Naive (or Parametric) End Users

- **Who they are:** The largest group of database users. Their main job revolves around constantly querying and updating the database.
- **Method:** They use **"Canned Transactions"**—standard queries and updates that have been carefully programmed and tested (often via mobile apps). They do not need to know how the DBMS works, only how to use the interface.
- **Examples:**
  - **Bank Tellers/Customers:** Checking balances or posting deposits.
  - **Reservation Agents:** Booking seats for airlines or hotels.
  - **Shipping Employees:** Scanning bar codes to update package status.
  - **Social Media Users:** Posting and reading updates.

C. Sophisticated End Users

- **Who they are:** Engineers, scientists, business analysts.
- **Usage:** They implement their own complex applications to meet specific requirements.
- **Skill Level:** They thoroughly familiarize themselves with the DBMS facilities.

D. Standalone Users

- **Who they are:** Individuals maintaining personal databases.
- **Method:** They use ready-made software packages with easy-to-use graphics or menus.
- **Example:** A user using a personal financial software package (like for taxes).

4. System Analysts and Application Programmers (Software Engineers)

These professionals build the tools for the "Naive" users.

- **System Analysts:** Determine the requirements of end users and develop specifications for "canned transactions."
- **Application Programmers:** Implement these specifications as code. They test, debug, document, and maintain the programs.
- **Requirement:** They must be familiar with the full range of DBMS capabilities to build effective tools.

Chapter 1.5: Workers Behind the Scene

These individuals work on the design, development, and operation of the DBMS software and the system environment. Unlike the "Actors on the Scene," these workers are typically **not interested in the actual content** (the data) stored in the database.

1. DBMS System Designers and Implementers

- **Role:** They design and build the DBMS software itself.
- **Complexity:** A DBMS is a highly complex software system comprised of many modules.
- **Key Modules Developed:**
  - Catalog implementation.
  - Query language processing.
  - Interface processing.
  - Accessing and buffering data.
  - Concurrency control.
  - Data recovery and security.
- **Integration:** They ensure the DBMS interfaces properly with other system software, like the **Operating System (OS)** and **compilers** for programming languages.

2. Tool Developers

- **Role:** They design and implement optional software packages (tools) that facilitate database design and performance.

- **Software Vendors:** These tools are often developed and marketed by independent software vendors and purchased separately from the DBMS.
- **Types of Tools:**
  - Database modeling and design tools.
  - Performance monitoring tools.
  - Graphical or natural language interfaces.
  - Prototyping and simulation tools.
  - Test data generation tools.

3. Operators and Maintenance Personnel

- **Role:** Also known as **system administration personnel**.
- **Responsibilities:** They manage the actual running and maintenance of the **hardware and software environment** required for the database system.

Summary of Roles

Role	Focus	Primary Goal
Actors on the Scene	The **Data**	Use/Administer the specific database content.
Workers Behind the Scene	The **System**	Build/Maintain the software and hardware infrastructure.

Chapter 1.6: Advantages of Using the DBMS Approach

Beyond the core characteristics discussed previously, a DBMS offers several advanced capabilities managed by the Database Administrator (DBA) to ensure efficiency and reliability.

1.6.1 Controlling Redundancy

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

(a)

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Brown	112	MATH2410	B

(b)

- **The Problem in File Processing:**
  - In traditional systems, different user groups (e.g., Accounting vs. Registration) often maintain their own separate files.
  - **Redundancy:** This leads to the same data (like a student's name) being stored multiple times.
  - **Consequences:**
    1. **Duplication of Effort:** Updates (like a name change) must be performed multiple times.
    2. **Wasted Space:** Storage is consumed by duplicate data.

3. **Inconsistency:** Files may become mismatched if an update is applied to one file but not another (e.g., one file has the correct birth date, another has an error).

- **The Database Solution (Normalization):**
  - Ideally, each logical data item is stored in **only one place**. This is called **Data Normalization**.
- **Controlled Redundancy (Denormalization):**
  - Sometimes, for performance, a design purposefully stores data in multiple places (e.g., storing `Student_name` in the `GRADE_REPORT` file to avoid looking it up every time).
  - The DBMS controls this by ensuring that if the name changes in one place, it automatically updates everywhere to prevent inconsistency.
  - *See Figure 1.6 for an example of inconsistent data caused by uncontrolled redundancy.*

1.6.2 Restricting Unauthorized Access

- Not all users should see all data (e.g., financial data like salaries should be confidential).
- **Security & Authorization Subsystem:**
  - The DBA uses this to create accounts and specify restrictions (read-only vs. update privileges).
  - Access is typically controlled via account numbers and passwords.
  - Parametric users may be restricted to using only specific "canned transactions".

1.6.3 Providing Persistent Storage for Program Objects

- **The Impedance Mismatch Problem:** Traditional database types often don't match the complex data structures (like classes or structs) used in programming languages like C++ or Java.
- **Object-Oriented Database Systems (OODBMS):**
  - These systems are compatible with object-oriented programming languages.
  - **Persistence:** Complex objects can be stored directly in the database and survive program termination, without the programmer needing to convert them to a file format manually.

1.6.4 Efficient Query Processing

- **Storage Structures:** Since databases are stored on disk, the DBMS uses specialized structures to speed up searching.
- **Indexes:** Auxiliary files (often Tree or Hash structures) used to locate records quickly.
- **Buffering/Caching:** The DBMS maintains parts of the database in main memory (RAM) to reduce slow disk access.
- **Query Optimization:** A specific module chooses the most efficient "execution plan" for a query.

1.6.5 Providing Backup and Recovery

- **Recovery Subsystem:** Restores the database to a correct state after hardware or software failure.
- **Atomicity:** If a complex transaction crashes halfway through, the system ensures the database returns to the state it was in *before* the transaction started.

1.6.6 Providing Multiple User Interfaces

- Because users define their skills differently (casual vs. expert), the DBMS provides various interfaces:
  - **GUIs (Graphical User Interfaces):** Menus and forms for standalone or mobile users.
  - **Query Languages:** For casual users.
  - **Programming Interfaces:** For application engineers.

1.6.7 Representing Complex Relationships

- A DBMS must be able to represent and manage data that is interrelated.

- *Example:* Connecting a Student to a Grade Report, and that Grade Report to a Course and Section.

1.6.8 Enforcing Integrity Constraints

- **Integrity Constraints:** Rules that the data must follow.
  - **Data Type:** e.g., Name must be a string of 30 characters; Grade cannot be 'Z'.
  - **Referential Integrity:** A record in one file must relate to a valid record in another (e.g., a Section must belong to a valid Course).
  - **Uniqueness (Key) Constraint:** Values must be unique (e.g., Course_number).
- **Business Rules:** Constraints derived from the specific meaning of the data in the "miniworld".

1.6.9 Inferencing, Rules, and Triggers

- **Deductive Database Systems:** Can define rules to infer *new* information from stored facts (e.g., determining which students are on probation based on complex criteria).
- **Triggers:** Rules activated by updates to a table that automatically perform additional operations (e.g., updating a second table when the first is changed).
- **Stored Procedures:** Involved procedures that become part of the database definition.

1.6.10 Additional Implications of the Database Approach

- **Enforcing Standards:** The DBA can enforce naming conventions, data formats, and report structures across the whole organization, facilitating cooperation between departments.
- **Reduced Development Time:** Creating new applications takes significantly less time (estimated 1/6th to 1/4th the time) because the database structure already exists.
- **Flexibility:** Modern DBMSs allow the database structure to evolve (e.g., adding new files) without breaking existing programs.
- **Availability of Up-to-Date Information:** As soon as one user updates data, it is immediately available to all other users (crucial for banking and reservations).
- **Economies of Scale:** Consolidating data allows the organization to buy fewer, more powerful servers rather than many small ones for each department, reducing overall costs.

Chapter 1.7: When Not to Use a DBMS

While a DBMS offers many powerful advantages, it is not always the best solution. There are specific situations where the costs outweigh the benefits.

1. The Costs (Overhead) of a DBMS

Using a DBMS introduces "overhead costs" that traditional file processing does not have. These costs come from:

- **High Initial Investment:** Significant cost for hardware, software, and training personnel.
- **Generality:** The system is designed to handle *anything*, which makes it less efficient than a tool designed to do *one specific thing*.
- **Feature Overhead:** The computational cost of providing security, concurrency control, recovery, and integrity functions (even if you don't need them).

2. When to Avoid a DBMS

It may be better to use **customized file processing** (writing your own code) in the following scenarios:

- **Simple & Static:** The application is simple, well-defined, and not expected to change at all.
- **Stringent Real-Time Requirements:** When the application needs extreme speed that the overhead of a general DBMS would slow down (e.g., real-time switching).
- **Embedded Systems:** Devices with limited storage capacity where a large, general-purpose DBMS simply wouldn't fit.
- **Single User:** If there is no need for multiple users to access the data simultaneously.

3. Industry Examples of Alternatives

Some industries use proprietary or custom data management instead of general-purpose DBMSs:

- **CAD (Computer-Aided Design):** Tools used by engineers often use proprietary file software optimized specifically for manipulating 3D objects and drawings.
- **Communication Switching (e.g., AT&T):** Used custom software optimized for speed and hierarchical data to route calls quickly.
- **GIS (Geographic Information Systems):** Often use custom data schemes to efficiently process maps, polygons, and contours.

Final Summary of Chapter 1

We have now covered the introduction to databases, including:

1. **Definitions:** Data vs. Information, DBMS, Database System.
2. **Characteristics:** Self-describing, program-data independence, multiple views, and sharing.
3. **Actors:** DBAs, Designers, End Users, and System Analysts.
4. **Workers:** Tool developers and system programmers.
5. **Advantages:** Controlling redundancy, security, persistent storage, and standards.
6. **Disadvantages:** When *not* to use a DBMS (overhead, real-time needs).

Chapter 2.1: Data Models, Schemas, and Instances

1. Data Abstraction

A fundamental characteristic of the database approach is **Data Abstraction**.

- **Definition:** The process of suppressing details of data organization and storage while highlighting essential features to improve understanding.
- **Goal:** To allow different users to perceive data at their preferred level of detail, hiding complex storage details.

2. Data Models

To achieve data abstraction, we use **Data Models**.

- **Definition:** A collection of concepts used to describe the **structure** of a database.
  - **Structure** includes: data types, relationships, and constraints.
- **Operations:** Most data models include basic operations for retrieving and updating data.
- **Behavior (Dynamic Aspect):** Modern models allow defining valid user-defined operations on the data (e.g., `COMPUTE_GPA` for a student object).
  - In relational models, this is often handled by **Stored Procedures**.

3. Categories of Data Models

Data models are categorized by the type of concepts they use:

A. High-Level or Conceptual Data Models

- **Target Audience:** Many users (closest to how humans perceive data).
- **Key Concepts:**
  - **Entity:** A real-world object (e.g., Employee, Project).
  - **Attribute:** A property of an entity (e.g., Name, Salary).
  - **Relationship:** An association between entities (e.g., an employee *works-on* a project).
- **Example:** The Entity-Relationship (ER) model.

B. Low-Level or Physical Data Models

- **Target Audience:** Computer specialists.
- **Focus:** Describes how data is stored on computer media (magnetic disks).
- **Key Concepts:** Record formats, record ordering, and **Access Paths** (search structures like **indexing** or **hashing** to find data efficiently).

C. Representational (Implementation) Data Models

- **Position:** Between High-Level and Low-Level models.
- **Features:** Concepts are understandable to end users but are close enough to storage organization to be implemented directly on a computer.
- **Structure:** They typically represent data using **records**, so they are often called **Record-Based Data Models**.

- **Examples:**
  - **Relational Data Model:** Widely used in commercial DBMSs (uses tables/relations).
  - **Legacy Models:** Network and Hierarchical models.

D. Other Models

- **Object Data Model:** A new family of higher-level implementation models closer to conceptual models.
- **Self-Describing Data Models:** Combine data description with data values (e.g., **XML**, **NoSQL** key-value stores). Useful for Big Data.

---

4. Schemas, Instances, and Database State

It is critical to distinguish between the *description* of the database and the *actual data*.

A. Database Schema (Intension)

- **Definition:** The **description** of the database structure.
- **Characteristics:**
  - Specified during design.
  - **Not expected to change frequently**.
  - Stored in the DBMS catalog as **Meta-data**.
- **Schema Diagram:** A visual display of the schema (e.g., record types, data items) but *not* the actual data instances.
  - *Note:* Diagrams often show names and structures but hide details like specific data types or complex constraints.

B. Database State (Extension/Snapshot)

- **Definition:** The actual **data** stored in the database at a particular moment in time.
- **Characteristics:**
  - Changes frequently (every time data is inserted, deleted, or updated).
  - Also called the set of **instances** or occurrences.
- **Valid State:** A state that satisfies the structure and constraints specified in the schema. The DBMS ensures every state is valid.

C. Schema Evolution

- Although schemas are static, requirements sometimes change.
- **Schema Evolution:** The process of applying changes to the schema (e.g., adding a `Date_of_birth` field) while the database is operational.

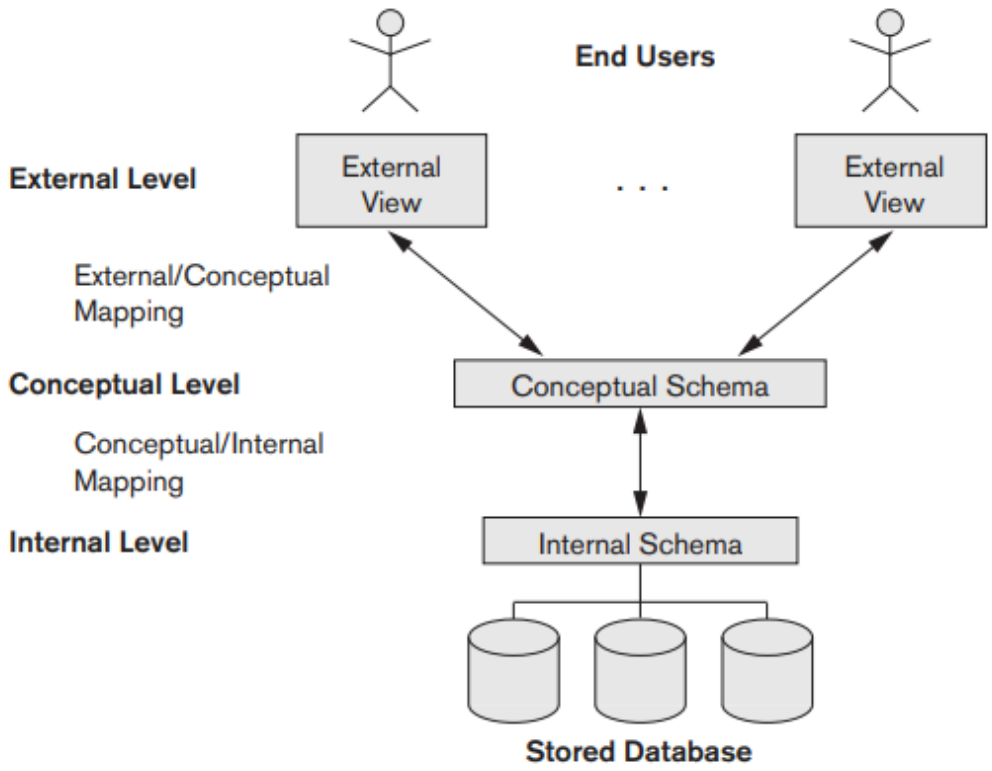
Summary Table: Schema vs. State

Feature	Database Schema (Intension)	Database State (Extension)
Definition	Description of database structure	The actual data at a specific moment
Frequency of Change	Infrequent (Design phase)	Frequent (Every update/insert)
Storage	Stored in Catalog (Meta-data)	Stored in Database Files
Example	STUDENT (Name, Number, Class)	('Smith', 17, 1)

Chapter 2.2: Three-Schema Architecture and Data Independence

This section defines the architecture used to support the key characteristics of the database approach: self-description, insulation between programs and data, and multiple user views.

1. The Three-Schema Architecture



The goal of this architecture is to clearly separate **user applications** from the **physical database**. It divides the system into three distinct levels:

A. Internal Level (Lowest Level)

- **Schema: Internal Schema**.
- **Function:** Describes the **physical storage structure** of the database.
- **Details:** Uses a physical data model to describe complete details of data storage and access paths (indexes, hashing).

B. Conceptual Level (Middle Level)

- **Schema: Conceptual Schema**.
- **Function:** Describes the structure of the **whole database** for the entire community of users.
- **Details:** Hides physical storage details. It focuses on entities, data types, relationships, user operations, and constraints.
- **Implementation:** Usually described using a representational data model (e.g., Relational Model).

C. External or View Level (Highest Level)

- **Schema: External Schemas** (or User Views).
- **Function:** Describes only the part of the database that a **specific user group** is interested in.
- **Details:** Hides the rest of the database from that group. Like the conceptual level, it is typically implemented using a representational data model.

Important Note: The actual data exists *only* at the physical (internal) level. The other levels are just descriptions (schemas). The DBMS must transform (map) requests from one level to another.

---

2. Mappings

- **Definition:** The process of transforming requests and results between the three levels.
- **Process:**
  1. A user sends a request based on their **External Schema**.
  2. DBMS transforms it into a request on the **Conceptual Schema**.
  3. DBMS transforms it again into a request on the **Internal Schema**.

4. Data is retrieved from storage and reformatted back up the chain to match the user's view.

- **Cost:** Mappings can be time-consuming, so some small DBMSs skip supporting external views.

---

3. Data Independence

Data Independence is the capacity to change the schema at one level **without having to change the schema at the next higher level**.

A. Logical Data Independence

- **Definition:** The capacity to change the **Conceptual Schema** without changing External Schemas or application programs.
- **Scenarios:**
  - Adding a new record type or data item (expanding the database).
  - Changing constraints.
  - Removing data that the external view doesn't use.
- **Mechanism:** Only the **view definition and mappings** need to be updated. The application code remains untouched.
- **Difficulty:** Harder to achieve than physical independence because it deals with structural changes.

B. Physical Data Independence

- **Definition:** The capacity to change the **Internal Schema** without changing the Conceptual Schema (and consequently, the External Schemas).
- **Scenarios:**
  - Creating new access structures (indexes) to improve performance.
  - Reorganizing physical files (splitting/merging records).
  - Changing storage encoding or compression.
- **Mechanism:** The application and conceptual schema remain unaware of these physical changes. Only the mapping to storage changes.
- **Availability:** Most databases and file systems support this to some degree.

Summary Table: Types of Data Independence

Type	Changes Occur In...	Does NOT Affect...	Example Scenario
Physical	Internal Schema (Storage)	Conceptual Schema	Adding an index to speed up a query.
Logical	Conceptual Schema (Structure)	External Schemas / Programs	Adding a "BirthDate" field to a Student table.

Chapter 2.3.1: DBMS Languages

Once a database is designed and a DBMS is chosen, specific languages are used to define the structure (schema) and manipulate the data. While older systems used distinct languages for each task, modern systems often combine them.

1. Types of Database Languages

A. Data Definition Language (DDL)

- **Purpose:** Used by the DBA and designers to define the database schemas.
- **Function:** The DDL compiler processes these statements to identify schema descriptions and stores them in the **DBMS catalog**.
- **Usage Contexts:**

    - **No Strict Separation:** In many DBMSs, DDL is used to define *both* the conceptual and internal schemas.
    - **Strict Separation:** If clear separation exists, DDL specifies the **conceptual schema** only.
    - **View Definition:** In most systems, DDL is also used to define external schemas (user views), though a separate **View Definition Language (VDL)** theoretically exists for this purpose.
    - *Note:* In relational DBMSs, **SQL** is used to define user views.

B. Storage Definition Language (SDL)

- **Purpose:** Used to specify the **internal schema** (physical storage structure).
- **Modern Usage:** Most modern relational DBMSs do *not* have a specific SDL language. Instead, they use a combination of functions and parameters to control indexing and storage mappings.
    - *Note:* SDL was removed from SQL to keep the language focused on conceptual and external levels.

C. Data Manipulation Language (DML)

- **Purpose:** Used to manipulate the database once it is populated. Operations include retrieval, insertion, deletion, and modification.
- **Integrated Language:** In current DBMSs (like SQL), DDL, VDL, and DML are not distinct languages but are combined into one comprehensive integrated language.

---

2. Categories of DML

There are two main types of Data Manipulation Languages:

A. High-Level or Nonprocedural DML (Set-at-a-Time)

- **Nature:** Declarative—it specifies *what* data to retrieve rather than *how* to retrieve it.
- **Capability:** Can specify complex operations concisely and retrieve **many records** in a single statement (hence "set-at-a-time" or "set-oriented").
- **Usage:**
    - **Interactive:** Entered directly from a terminal (Standalone Query Language).
    - **Embedded:** Placed within a general-purpose programming language. A **precompiler** extracts these statements for the DBMS to process.

B. Low-Level or Procedural DML (Record-at-a-Time)

- **Nature:** Must be embedded in a general-purpose programming language.
- **Capability:** Retrieves and processes **individual records** (or objects) one by one.
- **Usage:** Because it handles one record at a time, it requires programming constructs like **loops** to process a set of records.

---

3. Host Language vs. Data Sublanguage

- **Host Language:** The general-purpose programming language (e.g., C++, Java, Python) in which DML commands are embedded.
- **Data Sublanguage:** The DML itself when used inside a host language.
- **Query Language:** A high-level DML used in a standalone, interactive manner.

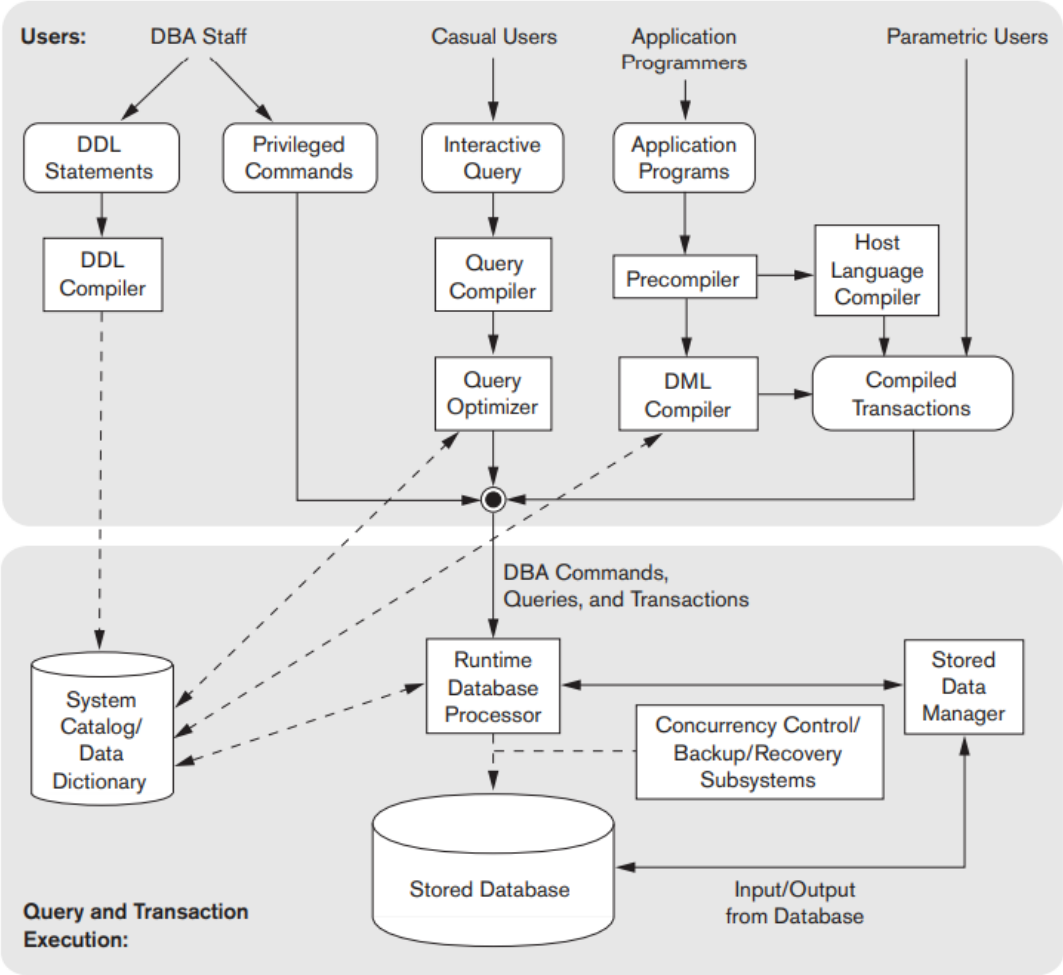
Chapter 2.4: The Database System Environment

A Database Management System (DBMS) is a complex software system. It interacts heavily with the computer's Operating System (OS) to manage hardware resources like memory and disk storage.

2.4.1 DBMS Component Modules

The architecture of a DBMS can be divided into two main sections: the **Top Part** (Users and Interfaces) and the **Lower Part** (Internal Storage and Processing).

1. Top Part: Users and Interfaces
This section handles the different types of users and how their requests are processed before execution.



- **DBA Staff (Database Administrators):**
  - **Role:** Define and tune the database structure.
  - **Tools:** They use **DDL (Data Definition Language)** and privileged commands.
  - **Process:** The **DDL Compiler** processes these definitions and stores the descriptions (meta-data) in the **DBMS Catalog**.
  - **Catalog:** Stores file names, sizes, data types, mapping information, and constraints. This is the central information source for other DBMS modules.
- **Casual Users:**
  - **Role:** Occasional users who need specific information.
  - **Tools:** Interactive Query Interfaces (mobile apps, forms, or query languages).
  - **Process:**
    1. **Query Compiler:** Parses and validates the query (checking syntax and verifying file names against the catalog).
    2. **Query Optimizer:** Rearranges operations to eliminate redundancy and selects efficient search algorithms. It consults the catalog for statistical data to generate an efficient execution plan.
- **Application Programmers:**
  - **Role:** Write programs in **Host Languages** (Java, C++, Python).
  - **Process:**
    1. **Precompiler:** Extracts database commands (DML) from the program code.
    2. **DML Compiler:** Compiles these commands into object code for database access.
    3. **Host Language Compiler:** Compiles the rest of the program.

        4. **Linker:** Combines everything into a **Canned Transaction** (executable code).
- **Parametric Users:**
    - **Role:** End users (e.g., bank tellers) who perform repetitive tasks.
    - **Method:** They execute **Canned Transactions** repeatedly, simply supplying new parameters (e.g., providing an account number and amount for a deposit).

---

2. Lower Part: Internal Processing and Storage
This section handles the actual execution of requests and management of data on the disk.

- **Runtime Database Processor:**
    - The engine that executes:
        1. Privileged commands from the DBA.
        2. Executable query plans from the optimizer.
        3. Canned transactions with runtime parameters.
    - It manages data transfer and often handles **buffer management** (deciding what data stays in main memory RAM).
- **Stored Data Manager:**
    - Controls access to the actual data stored on disk (both the database and the catalog).
    - It interacts with the **Operating System (OS)** to schedule low-level Input/Output (read/write) operations.
- **Concurrency Control and Backup/Recovery:**
    - These vital systems are integrated into the Runtime Database Processor to ensure transactions are safe (backup) and do not conflict with each other (concurrency).

---

3. Infrastructure and System Interaction
- **Disk Storage:** The Database and the Catalog are stored here.
- **Buffer Management:**
    - Since disk access is slow, keeping frequently used data in main memory (buffering) is critical for performance.
    - Some DBMSs have their own module for this, while others rely on the OS.
- **Client-Server Architecture:**
    - **Database Server:** The machine holding the database and DBMS software.
    - **Client Computer:** The machine running the user interface or application.
    - **Application Server:** A middle layer often used between the client and the database server.

Chapter 2.6: Classification of Database Management Systems

DBMSs can be classified using several criteria: Data Model, Number of Users, Distribution, Cost, and Purpose.

1. Classification by Data Model

This is the main criterion for classifying a DBMS. It refers to the underlying structure used to organize the data.

- **Relational Data Model (SQL Systems):**
    - **Structure:** Represents data as a collection of **tables** (relations), where each table is stored as a separate file.
    - **Language:** Uses **SQL** (a high-level query language).

    - **Usage:** The dominant model in commercial systems.
- **Object Data Model:**
    - **Structure:** Defines database in terms of **objects**, properties, and operations.
    - **Organization:** Objects with the same structure belong to a **class**, and classes are organized into hierarchies (acyclic graphs).
    - **Methods:** Operations are specified as predefined procedures called **methods**.
- **Object-Relational DBMSs:**
    - **Definition:** Relational systems that have evolved to incorporate object database concepts.
- **Big Data / NoSQL Systems:**
    - Used for managing massive amounts of data, often using one of four specific models:
        1. **Key-Value Model:** Associates a unique key with each value (record/object) for very fast access.
        2. **Document Model:** Based on **JSON**; stores data as documents that resemble complex objects.
        3. **Graph Model:** Stores objects as **nodes** and relationships as **edges** in a graph.
        4. **Column-Based Model:** Stores columns of data clustered together on disk for fast access and allows multiple data versions.
- **XML Data Model:**
    - **Structure:** Tree-structured model using hierarchical tags and nesting.
    - **Usage:** Standard for Web data exchange. Some "Native XML DBMSs" exist, but many relational systems now also support XML interfaces.
- **Legacy Data Models:**
    - **Network Model:** Represents data as record types and uses **1:N relationships** (set types) via pointers. Associated with the **CODASYL DBTG** standard and COBOL extensions.
    - **Hierarchical Model:** Represents data as strict **hierarchical tree structures**. Dominated the market (1965–1985) with systems like **IMS** and its language **DL/1**.

---

2. Classification by Number of Users

- **Single-User Systems:** Support only one user at a time. Mostly used with personal computers (PCs).
- **Multiuser Systems:** Support multiple users accessing the database concurrently. This includes the majority of DBMSs.

---

3. Classification by Distribution (Number of Sites)

- **Centralized DBMS:** Data and DBMS software reside entirely at a **single computer site**.
- **Distributed DBMS (DDBMS):**
    - **Definition:** The database and software are distributed over many sites connected by a network.
    - **Massively Distributed:** Big Data systems often span hundreds of sites with data **replicated** (copied) to handle failures.
    - **Types of DDBMS:**
        - **Homogeneous:** Uses the **same** DBMS software at all sites.
        - **Heterogeneous:** Uses **different** DBMS software at different sites.
        - **Federated (Multidatabase):** Uses **middleware** to access several autonomous, preexisting databases that are loosely coupled.

---

4. Classification by Cost

- **Open Source:** Free products (e.g., MySQL, PostgreSQL) often supported by third-party vendors selling services.

- **Personal Versions:** Low cost (under 100) or free trial versions with reasonable functionality.
- **Giant Systems:**
    - Sold in **modular form** (extra cost for replication, mobile support, data mining, etc.).
    - **Site Licenses:** Allow unlimited use at one location.
    - **Seat Licenses:** Limit the number of concurrent users.
    - **Cost:** Installation and annual maintenance for large systems can cost millions of dollars.

---

5. Other Classifications

- **By Access Path:** Some systems are classified by their storage method, such as **Inverted File Structures**.
- **By Purpose:**
    - **General Purpose:** Can be used for many types of applications.
    - **Special Purpose:** Designed for a specific application where performance is critical (e.g., airline reservations, telephone directories).
        - **OLTP (Online Transaction Processing):** Special systems designed to handle large numbers of concurrent transactions without delay.

Chapter 3.1: Using High-Level Conceptual Data Models for Database Design

Designing a database is a structured process that moves from understanding user needs to actually building the system. This process is illustrated in **Figure 3.1**, which separates the design into two parallel tracks: **Data Design** (structuring the information) and **Functional Design** (designing the software/applications).

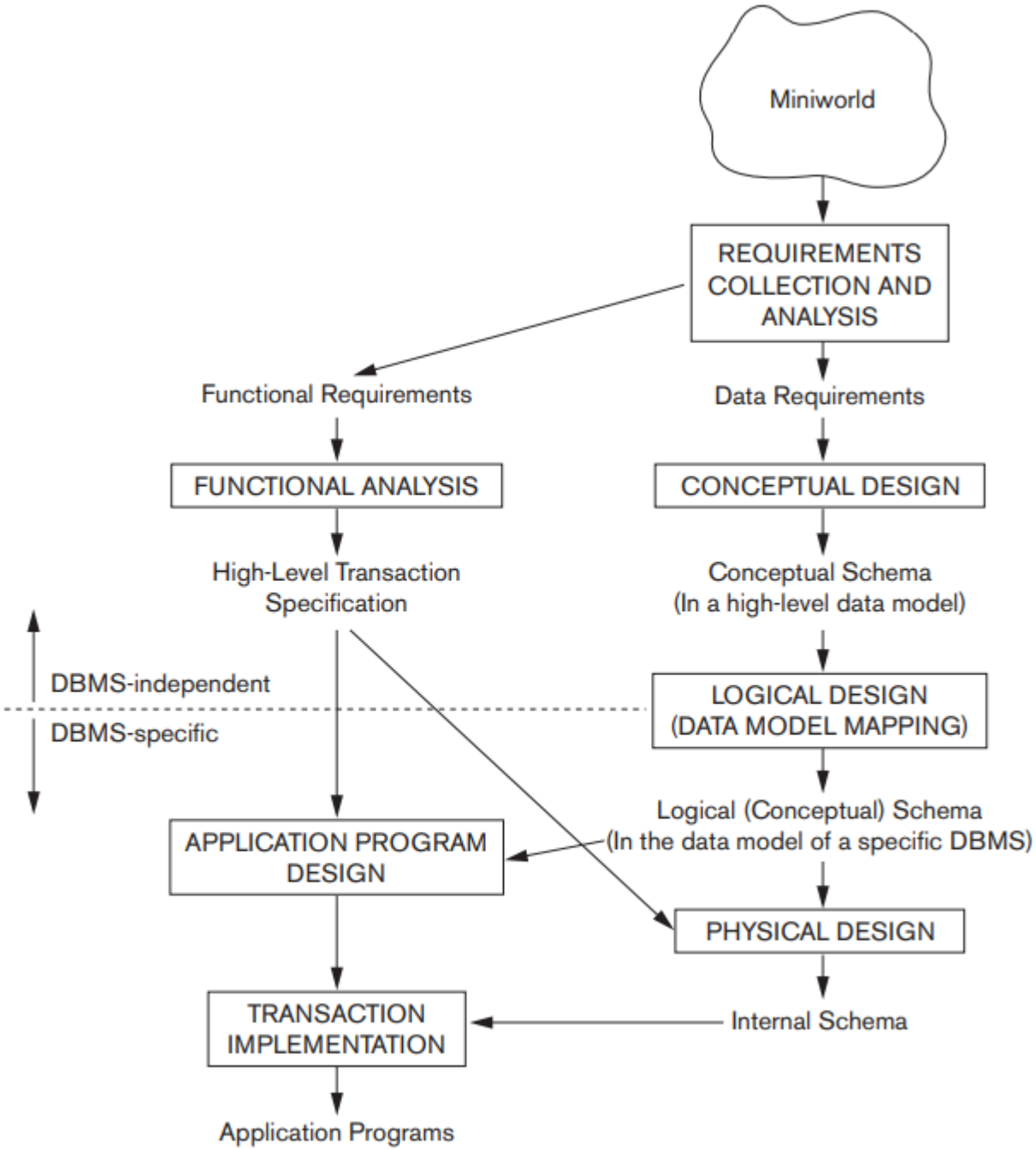


Figure 3.1
A simplified diagram to illustrate the main phases of database design.

Phase 1: Requirements Collection and Analysis

Before any design begins, designers must understand what is needed.

- **Process:** Designers interview prospective users to understand their needs.
- **Goal:** To produce a concise, detailed, and complete set of **Users' Requirements**.
- **Two Types of Requirements:**
  1. **Data Requirements:** What data needs to be stored?
  2. **Functional Requirements:** What operations (transactions) will users perform? (e.g., retrievals, updates).
- *Note:* Software engineering techniques like data flow diagrams or sequence diagrams are often used here to specify these functional requirements.

Phase 2: Conceptual Design

Once requirements are gathered, the next step is to create a high-level map of the database.

- **Conceptual Schema:** A concise description of the data requirements using a **High-Level Conceptual Data Model**.
- **Content:** It describes **entities**, **relationships**, and **constraints**.
- **Key Benefit:** It does *not* include implementation details (like storage specs). This makes it easy for non-technical users to understand and verify that their requirements are met.
- **Validation:** Designers check the schema to ensure all identified functional requirements (queries and updates) can be performed on it.

Phase 3: Logical Design (Data Model Mapping)

This phase bridges the gap between the abstract design and the actual software.

- **Process:** The high-level conceptual schema is transformed into an **Implementation Data Model** supported by a commercial DBMS (most commonly the **Relational/SQL model**).
- **Result:** A database schema in the specific implementation model of the chosen DBMS.
- **Automation:** This step is often automated or semi-automated by database design tools.

Phase 4: Physical Design

The final data design phase deals with actual storage.

- **Process:** Specifying the internal storage structures and physical design parameters for the database files.
- **Details Specified:** File organizations, **indexes**, and **access paths** to ensure efficiency.

Parallel Activity: Application Design

While the data is being designed (Phases 2-4), the application software is also being built (see the left side of Figure 3.1).

- **Functional Analysis:** Analyzing the functional requirements to create high-level transaction specifications.
- **Implementation:** Application programs are designed and implemented as **database transactions**.

Summary of the Design Flow

1. **Real World** (Miniworld) \rightarrow **Requirements**
2. **Requirements** \rightarrow **Conceptual Schema** (High-Level)
3. **Conceptual Schema** \rightarrow **Logical Schema** (Implementation/SQL)
4. **Logical Schema** \rightarrow **Physical Schema** (Storage/Indexes)

Here are the detailed notes for **Sections 3.2 and 3.3**, covering the sample "COMPANY" database application and the fundamental concepts of Entities and Attributes in the Entity-Relationship (ER) model.

Chapter 3.2: A Sample Database Application (COMPANY)

To illustrate ER model concepts, the text introduces a hypothetical **COMPANY** database. The design begins with a set of requirements collected from the "miniworld" (the part of the real world being modeled).

Data Requirements for COMPANY

1. **Departments:**
    - The company is organized into departments.
    - Each department has a **unique name**, a **unique number**, and a specific **manager** (an employee).
    - The database tracks the **start date** when the manager began managing the department.
    - A department can have **multiple locations**.
2. **Projects:**
    - Departments control projects.
    - Each project has a **unique name**, a **unique number**, and a **single location**.
3. **Employees:**

    - Data stored: **Name**, **Social Security Number (SSN)**, **Address**, **Salary**, **Sex**, and **Birth Date**.
    - Assignments: An employee belongs to **one** department but may work on **several** projects (which may be controlled by different departments).
    - Tracking: The system tracks **hours per week** worked on each project.
    - Supervision: The system tracks the **direct supervisor** of each employee.
4. **Dependents:**
    - For insurance purposes, the system tracks dependents.
    - Data stored: **First Name**, **Sex**, **Birth Date**, and **Relationship** to the employee.

Chapter 3.3: Entity Types, Entity Sets, Attributes, and Keys

The ER model describes data using three main concepts: **Entities**, **Relationships**, and **Attributes**.

3.3.1 Entities and Attributes

1. Entities

- **Definition:** An entity is a "thing" or object in the real world with an independent existence.
- **Types of Existence:**
    - **Physical Existence:** A person, car, house, or employee.
    - **Conceptual Existence:** A company, job, or university course.

2. Attributes

- **Definition:** The particular properties that describe an entity.
- **Values:** Each entity has a specific value for each of its attributes, which becomes the major part of the stored data.
    - *Example:* An EMPLOYEE entity might have attributes Name, Address, Age, and Salary.

3. Classification of Attributes

A. Simple (Atomic) vs. Composite Attributes

- **Composite Attributes:** Can be divided into smaller subparts with independent meanings.
    - *Example:* Address can be broken down into Street_address, City, State, and Zip.
    - *Hierarchy:* Components can be further divided (e.g., Street_address divides into Number, Street, and Apartment_number).
    - *Usage:* Useful when users sometimes reference the whole (Address) and sometimes the part (Zip Code).
- **Simple (Atomic) Attributes:** Attributes that are not divisible.

B. Single-Valued vs. Multivalued Attributes

- **Single-Valued:** An attribute that has a single value for a particular entity.
    - *Example:* Age (a person only has one age).
- **Multivalued:** An attribute that can have a set of values for the same entity.
    - *Example:* Colors for a two-tone car, or College_degrees for a person.
    - *Constraints:* These attributes may have lower and upper bounds (e.g., a car can have at most two colors).
    - *Notation:* Displayed between curly braces { } in text representations (e.g., {Colors}).

C. Stored vs. Derived Attributes

- **Stored Attribute:** An attribute explicitly saved in the database.
    - *Example:* Birth_date.
- **Derived Attribute:** An attribute whose value is calculated from related attribute values.
    - *Example:* Age is derived from Birth_date and the current date.
    - *Example:* Number_of_employees for a Department is derived by counting the employees related to that department.

D. NULL Values

A special value used when an attribute does not have a valid actual value. It has three distinct meanings:

1. **Not Applicable:** The attribute does not apply to this entity (e.g., Apartment_number for a single-family house, or College_degrees for someone without one).

2. **Unknown (Missing):** The value exists but is not currently recorded (e.g., a person has a height, but it is not listed).
3. **Unknown (Existence Uncertain):** We do not know if the value exists (e.g., we don't know if a person has a home phone number).

E. Complex Attributes

- **Definition:** Attributes that nest composite and multivalued attributes arbitrarily.
- **Notation:** Uses parentheses ( ) for grouping components and braces { } for multiple values.
    - *Example:* Address_phone attribute where a person has multiple residences, and each residence has an address and multiple phones.

Chapter 3.3.2: Entity Types, Entity Sets, Keys, and Value Sets

This section refines the concept of "entities" by distinguishing between the *type* (the definition) and the *set* (the collection of actual data). It also introduces unique identifiers (Keys) and the valid range of values (Domains).

1. Entity Types vs. Entity Sets

- **Entity Type (Intension/Schema):**
    - **Definition:** Defines a collection of entities that share the **same attributes**.
    - **Description:** Described by its name and its list of attributes.
    - **Example:** EMPLOYEE is an entity type described by attributes Name, Age, and Salary.
    - **Diagram Notation:** Represented as a **rectangular box** enclosing the name.
        - Attribute names are enclosed in **ovals** and attached by straight lines.
        - Multivalued attributes use **double ovals**.
- **Entity Set (Extension/Collection):**
    - **Definition:** The collection of *all* actual entities of a particular type in the database at a specific point in time.
    - **Example:** The specific records for "John Smith", "Fred Brown", and "Judy Clark" make up the entity set for EMPLOYEE.
    - **Naming:** Often referred to by the same name as the entity type (e.g., "EMPLOYEE" refers to both the definition and the current data), though object-oriented models sometimes separate them.

---

2. Key Attributes (Uniqueness Constraint)

- **Definition:** An attribute (or combination of attributes) whose values are **distinct** for each individual entity in the set.
- **Purpose:** Used to identify each entity uniquely.
- **Examples:**
    - Name is a key for the COMPANY entity type (no two companies have the same name).
    - SSN is a typical key for PERSON.
- **Composite Key:** A key formed by a combination of multiple attributes.
    - **Minimal:** All components must be necessary for uniqueness (no superfluous attributes).
    - *Example:* Registration for a car is a key composed of State and Number.
- **Diagram Notation:** The key attribute's name is **underlined** inside the oval.
- **Multiple Keys:** An entity type can have more than one key.
    - *Example:* A CAR entity might have Vehicle_id (VIN) as one key and Registration (State + Number) as a second, separate key.
- **Weak Entity Type:** An entity type that has **no key** of its own (discussed later in Section 3.5).

---

3. Value Sets (Domains)

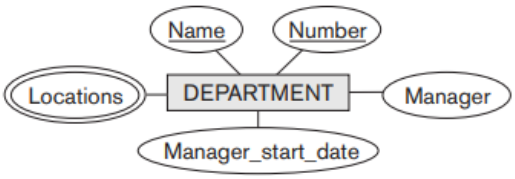
- **Definition:** The set of valid values that can be assigned to a simple attribute.
- **Examples:**
  - Age for an employee might have a value set of integers between 16 and 70.
  - Name might be strings of alphabetic characters.
- **Visibility:** Typically **not displayed** in basic ER diagrams but specified in the design tools or detailed documentation (similar to data types in programming: integer, string, boolean).
- **Mathematical Definition:** An attribute A is a function mapping an entity set E to the power set of values P(V): A : E \rightarrow P(V).
  - Single-valued attribute: Returns a singleton set (one value).
  - NULL value: Returns an empty set.

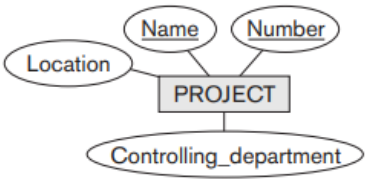
Chapter 3.3.3: Initial Conceptual Design of the COMPANY Database

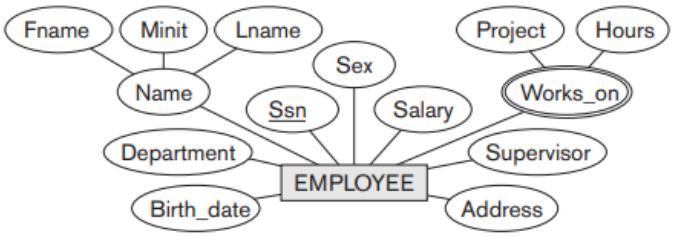
Based on the requirements gathered in Section 3.2, we can now define the initial entity types and their attributes. This design is "preliminary" because it focuses only on entities; relationships will be refined in later sections.

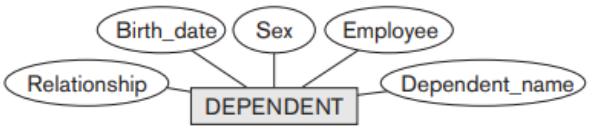
1. Identified Entity Types

From the requirements, four main entity types are identified:









A. DEPARTMENT Entity

- **Attributes:**
  - Name and Number: Defined as **key attributes** because each is unique.
  - Locations: A **multivalued attribute** (a department can have multiple locations).

    - Manager and Manager_start_date: Attributes tracking the manager.

B. PROJECT Entity

- **Attributes:**
    - Name and Number: Defined as **key attributes** (unique identifiers).
    - Location: A single-valued attribute (unlike Department locations).
    - Controlling_department: Tracks which department owns the project.

C. EMPLOYEE Entity

- **Attributes:**
    - Name: Modeled as a **composite attribute** (First_name, Middle_initial, Last_name) based on user needs.
    - Ssn: The key attribute (Social Security Number).
    - Address, Salary, Sex, Birth_date.
    - Department: Tracks which department the employee works for.
    - Supervisor: Tracks the employee's direct manager.
    - Works_on: A **multivalued composite attribute** containing (Project, Hours). This handles the requirement that an employee can work on several projects with specific hours for each.
        - *Note:* This specific attribute will later be refined into a relationship in the next design phase.

D. DEPENDENT Entity

- **Attributes:**
    - Employee: Links the dependent to the employee.
    - Dependent_name, Sex, Birth_date, Relationship.

Design Considerations

- **Composite vs. Simple:** Deciding whether to make Name or Address composite depends on user consultation. If users need to search by "Last Name" specifically, it must be composite. In this design, Name is composite, but Address is kept simple.
- **Placement of Attributes:** The Works_on data (Project + Hours) could theoretically belong to either the EMPLOYEE or the PROJECT. In this initial design, it is placed on the EMPLOYEE.

Chapter 3.4: Relationship Types, Sets, Roles, and Constraints

In the previous section, we identified entities like EMPLOYEE and DEPARTMENT. However, attributes like Manager (in Department) or Supervisor (in Employee) are actually references to other entities. In the ER model, these references are explicitly modeled as **Relationships** rather than attributes.

3.4.1 Relationship Types, Sets, and Instances

- **Relationship Type (R):**
    - **Definition:** Defines a set of associations (or a relationship set) among n entity types.
    - **Notation:** Displayed as a **diamond-shaped box** in ER diagrams, connected by straight lines to the participating entity types.
- **Relationship Set:**
    - **Definition:** The mathematical set of all actual relationship instances. It is a subset of the Cartesian product of the entity sets: E_1 \times E_2 \times \dots \times E_n.
- **Relationship Instance (r_i):**
    - **Definition:** A specific association of entities (e.g., Employee e_1 works for Department d_1).
    - **Example:** In the WORKS_FOR relationship, an instance connects one EMPLOYEE entity to one DEPARTMENT entity.

3.4.2 Relationship Degree, Role Names, and Recursive Relationships

1. Degree of a Relationship Type

The **degree** is the number of participating entity types.

- **Binary Relationship:** Degree 2 (connects two entity types). This is the most common type (e.g., WORKS_FOR connects Employee and Department).

- **Ternary Relationship:** Degree 3 (connects three entity types).
    - *Example:* SUPPLY relationship connecting **Supplier**, **Part**, and **Project** (a supplier supplies a specific part to a specific project).

2. Relationships as Attributes

You can often view binary relationships as attributes:

- **Perspective 1:** Department is an attribute of Employee (referencing the department they work for).
- **Perspective 2:** Employees is a multivalued attribute of Department (referencing the set of workers).
- **Constraint:** If both are represented, they must be **inverses** of each other.

3. Role Names and Recursive Relationships

- **Role Name:** Explains what part a participating entity plays in the relationship (e.g., in WORKS_FOR, the Employee plays the role of "Worker" and Department plays "Employer").
- **Recursive (Self-Referencing) Relationship:**
    - **Definition:** A relationship type where the **same entity type** participates more than once in different roles.
    - **Necessity:** Role names are *essential* here to distinguish meaning.
    - *Example:* The SUPERVISION relationship relates an EMPLOYEE (Supervisor role) to another EMPLOYEE (Supervisee/Subordinate role).

Chapter 3.4.4: Attributes of Relationship Types

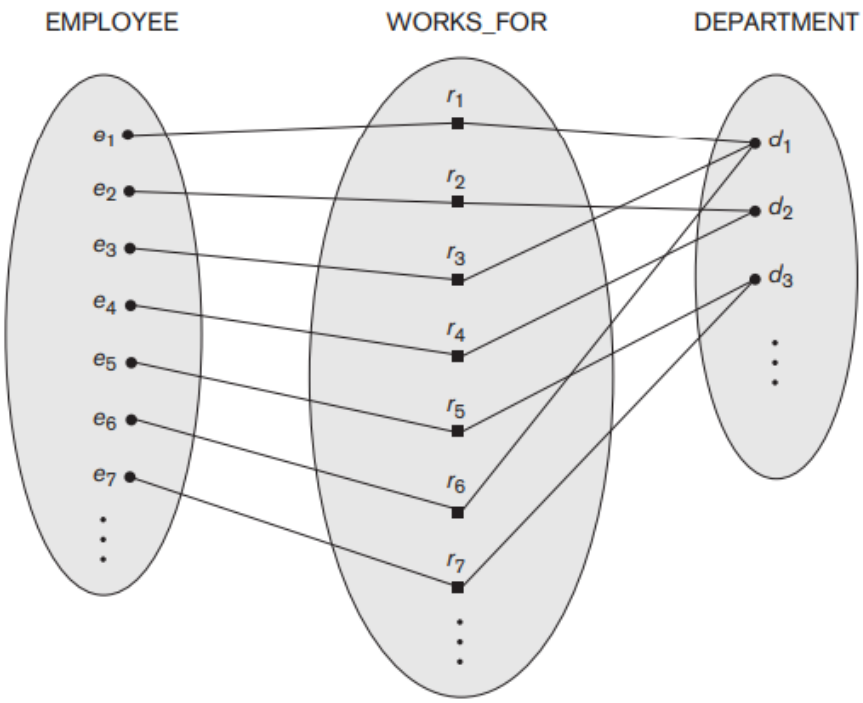
Just as entity types have attributes (properties), **Relationship Types** can also have attributes to describe properties of the association itself.

1. Defining Relationship Attributes

- **Concept:** Relationships often need to store data that doesn't belong strictly to one entity or the other, but rather to the *interaction* between them.
- **Examples:**
    - **Hours:** In a WORKS_ON relationship between EMPLOYEE and PROJECT, the number of hours worked belongs to the specific assignment of that employee to that project.
    - **Start_date:** In a MANAGES relationship between EMPLOYEE and DEPARTMENT, the date the management started describes the relationship instance.

2. Placement/Migration of Relationship Attributes

Depending on the "Cardinality Ratio" (1:1, 1:N, or M:N) of the relationship, these attributes can sometimes be moved ("migrated") to one of the participating entity types to simplify the design.



A. 1:1 Relationship Types

- **Rule:** The attribute can be migrated to **either** of the participating entity types.
- **Reasoning:** Since every entity participates in at most one relationship instance, the attribute value is unique to that entity.
- **Example:** For the MANAGES relationship (1 Employee manages 1 Department), the Start_date attribute can be stored on the EMPLOYEE (Manager) record **or** the DEPARTMENT record.

B. 1:N (One-to-Many) Relationship Types

- **Rule:** The attribute can be migrated **only** to the entity type on the **N-side** (the "Many" side) of the relationship.
- **Reasoning:**
  - The entity on the N-side participates in at most *one* relationship instance.
  - The entity on the 1-side participates in *many* instances, so it cannot store a single value for the attribute.
- **Example:** In WORKS_FOR (1 Department has N Employees), a Start_date attribute can be moved to the EMPLOYEE entity because each employee works for only one department. It cannot be on DEPARTMENT because a department has many start dates (one for each employee).

C. M:N (Many-to-Many) Relationship Types

- **Rule:** The attribute **must** remain specified as a **relationship attribute**.
- **Reasoning:** The value is determined by the **combination** of participating entities, not by any single entity.
- **Example:** In WORKS_ON (Employees work on many Projects; Projects have many Employees), the Hours attribute depends on *which* employee is working on *which* project. It cannot be stored on EMPLOYEE (which project's hours?) or PROJECT (which employee's hours?).

Summary Table: Attribute Migration

Relationship Type	Where can the Attribute go?
1:1 (One-to-One)	**Either** Entity Type
1:N (One-to-Many)	**Only** the N-side Entity Type
M:N (Many-to-Many)	**Must stay** on the Relationship

Chapter 3.4.3: Constraints on Binary Relationship Types

Relationships usually have rules (constraints) derived from the "miniworld" that limit which entities can be related. These are collectively called **Structural Constraints**.

1. Cardinality Ratios (Maximum Participation)

The cardinality ratio specifies the **maximum** number of relationship instances that an entity can participate in.

- **1:N (One-to-Many):**
  - **Definition:** An entity on one side can relate to *any number* (N) of entities on the other side, but the reverse is not true.
  - **Example:** WORKS_FOR relationship.
    - A **Department** can have many **Employees** (N).
    - An **Employee** works for at most one **Department** (1).
  - *See Figure 3.9 for the visual representation of these instances.*
- **1:1 (One-to-One):**
  - **Definition:** Each entity on both sides can relate to at most one entity on the other side.
  - **Example:** MANAGES relationship.
    - An **Employee** manages at most one **Department**.

        - A **Department** has at most one **Manager**.
- **M:N (Many-to-Many):**
    - **Definition:** Entities on both sides can relate to multiple entities on the other side.
    - **Example:** WORKS_ON relationship.
        - An **Employee** works on several **Projects**.
        - A **Project** has several **Employees**.
- **ER Diagram Notation:** Represented by displaying 1, M, or N on the relationship lines.

2. Participation Constraints (Minimum Participation)

The participation constraint (or existence dependency) specifies the **minimum** number of relationship instances an entity *must* participate in.

- **Total Participation (Existence Dependency):**
    - **Definition:** Every entity in the set **must** be related to another entity via this relationship.
    - **Example:** Every EMPLOYEE must work for a DEPARTMENT. An employee cannot exist in the database without a department.
    - **Notation:** Displayed as a **Double Line** connecting the entity to the relationship diamond.
- **Partial Participation:**
    - **Definition:** Only *some* entities in the set are related to another entity.
    - **Example:** Not every EMPLOYEE manages a DEPARTMENT. Only a few do.
    - **Notation:** Displayed as a **Single Line**.

Chapter 3.4.4: Attributes of Relationship Types

Relationship types can also have attributes, similar to entities. The handling of these attributes depends on the cardinality ratio.

1. Examples of Relationship Attributes

- **Hours:** In the WORKS_ON relationship, the attribute Hours belongs to the specific combination of an Employee and a Project (not just the employee, and not just the project).
- **Start_date:** In the MANAGES relationship, the date represents when that specific management assignment began.

2. Attribute Migration Rules

To simplify the database design, attributes of relationships can sometimes be moved ("migrated") to the participating entities.

Relationship Ratio	Migration Rule	Reason
1:1 (One-to-One)	Can migrate to **either** entity type.	Since both sides participate at most once, the attribute is unique to either entity (e.g., Start_date can go on EMPLOYEE or DEPARTMENT).
1:N (One-to-Many)	Can migrate **only** to the **N-side** entity.	The N-side entity participates in only one relationship instance, so it can hold the value. The 1-side entity participates in many, so it can't hold a single value (e.g., Start_date for WORKS_FOR goes on EMPLOYEE, not DEPARTMENT).
M:N (Many-to-Many)	**Cannot migrate.**	Must remain as a relationship attribute because the value depends on the *combination* of entities (e.g., Hours worked depends on both the Employee and the Project).

Chapter 3.5: Weak Entity Types

- **Definition:** An entity type that **does not have a key attribute** of its own.
- **Identification:** Entities of this type are identified by combining:
  1. The key of another entity type (called the **Owner** or **Identifying Entity Type**).
  2. A **Partial Key** (or Discriminator) from the weak entity itself.
- **Constraints:**
  - **Total Participation:** A weak entity type *always* has a total participation constraint (existence dependency) with its owner. A weak entity cannot exist without an owner.
  - *Note:* Not all existence dependencies imply a weak entity. For example, a DRIVER_LICENSE cannot exist without a PERSON, but if it has a unique License_number, it is a **Strong Entity Type**.
- **Example: DEPENDENT Entity**
  - **Owner:** EMPLOYEE.
  - **Attributes:** Name, Birth_date, Sex, Relationship.
  - **Problem:** Two different employees might both have a dependent named "Alice". "Alice" is not unique across the whole database.
  - **Solution:** A dependent is identified by the Employee's key *plus* the dependent's Name. Thus, Name is the **Partial Key**.
- **ER Diagram Notation:**
  - **Weak Entity:** Double Rectangle.
  - **Identifying Relationship:** Double Diamond.
  - **Partial Key:** Underlined with a **dashed/dotted line**.

Chapter 3.6: Refining the ER Design for COMPANY

We now convert the initial design (Section 3.3) into a refined schema by changing specific attributes into proper **Relationship Types**.

1. **MANAGES (Employee ↔ Department):**
   - **Type:** 1:1 relationship.
   - **Constraint:** Partial participation for Employee (not everyone manages); Total participation for Department (must have a manager).
   - **Attribute:** Start_date.
2. **WORKS_FOR (Department ↔ Employee):**
   - **Type:** 1:N relationship.
   - **Constraint:** Total participation for both (Employee must work for Dept; Dept must have Employees).
3. **CONTROLS (Department ↔ Project):**
   - **Type:** 1:N relationship.
   - **Constraint:** Total for Project (must be controlled); Partial for Department.
4. **SUPERVISION (Employee ↔ Employee):**
   - **Type:** 1:N recursive relationship.
   - **Constraint:** Partial for both (not everyone is a supervisor; not everyone has a supervisor—e.g., the CEO).
5. **WORKS_ON (Employee ↔ Project):**
   - **Type:** M:N relationship.
   - **Constraint:** Total participation for both.
   - **Attribute:** Hours.
6. **DEPENDENTS_OF (Employee ↔ Dependent):**
   - **Type:** 1:N Identifying Relationship.
   - **Constraint:** Partial for Employee; Total for Dependent.

Chapter 3.7: ER Diagrams, Naming, and Design Issues

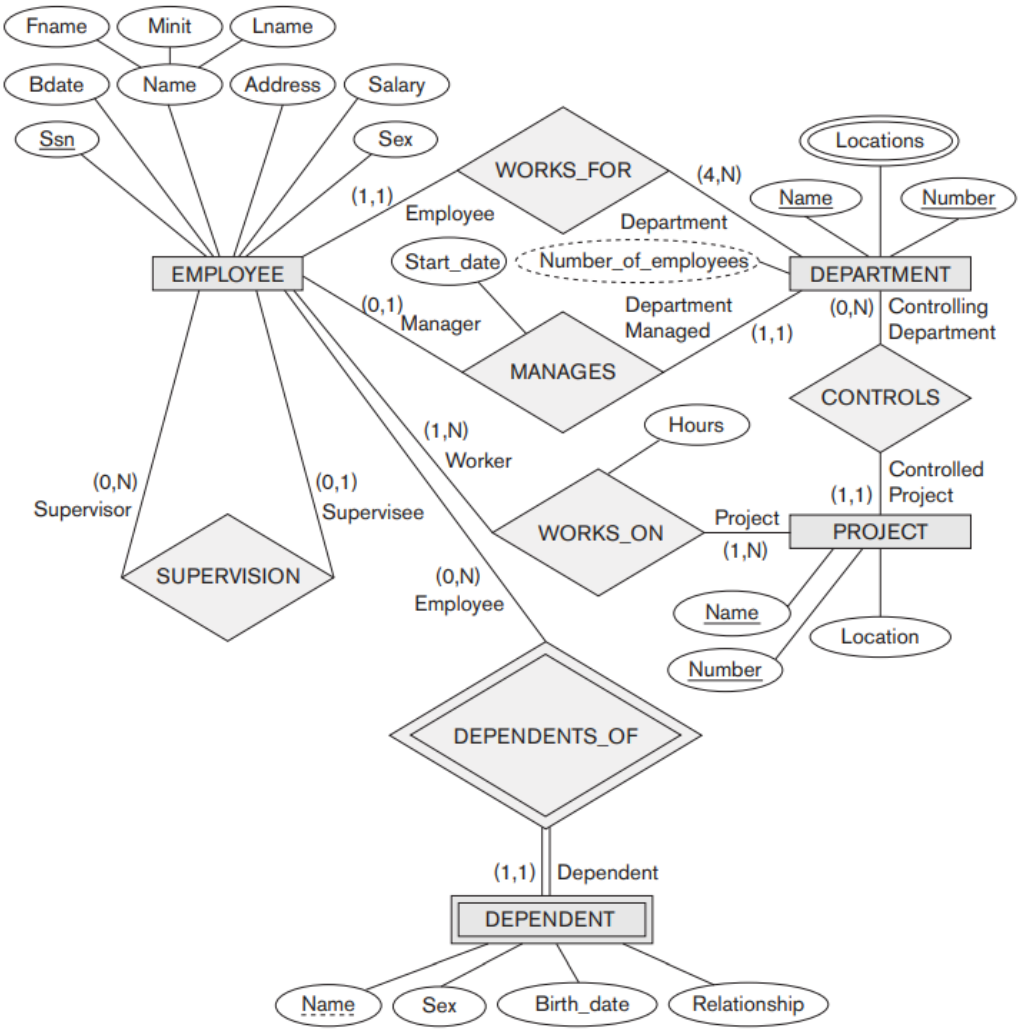


Figure 3.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

3.7.1 Summary of Notation

- **Strong Entity:** Rectangular Box.
- **Weak Entity:** Double Rectangular Box.
- **Relationship:** Diamond Box.
- **Identifying Relationship:** Double Diamond Box.
- **Attributes:** Ovals.
  - **Key:** Underlined.
  - **Partial Key:** Dotted/Dashed Underline.
  - **Multivalued:** Double Oval.
  - **Derived:** Dotted Oval.
- **Structural Constraints:**
  - **Cardinality:** 1, M, or N on lines.
  - **Participation:** Single line (Partial) or Double line (Total).

3.7.2 Proper Naming

- **Entity Types:** Singular nouns (e.g., EMPLOYEE, not EMPLOYEES).
- **Relationship Types:** Verbs (e.g., WORKS_FOR, MANAGES).
- **Readability:** Names should be chosen so diagrams read left-to-right or top-to-bottom.
  - *Example:* DEPENDENTS_OF reads bottom-to-top, which is an exception. Renaming it to HAS_DEPENDENTS would fix this flow.

3.7.3 Design Choices (Iterative Refinement)

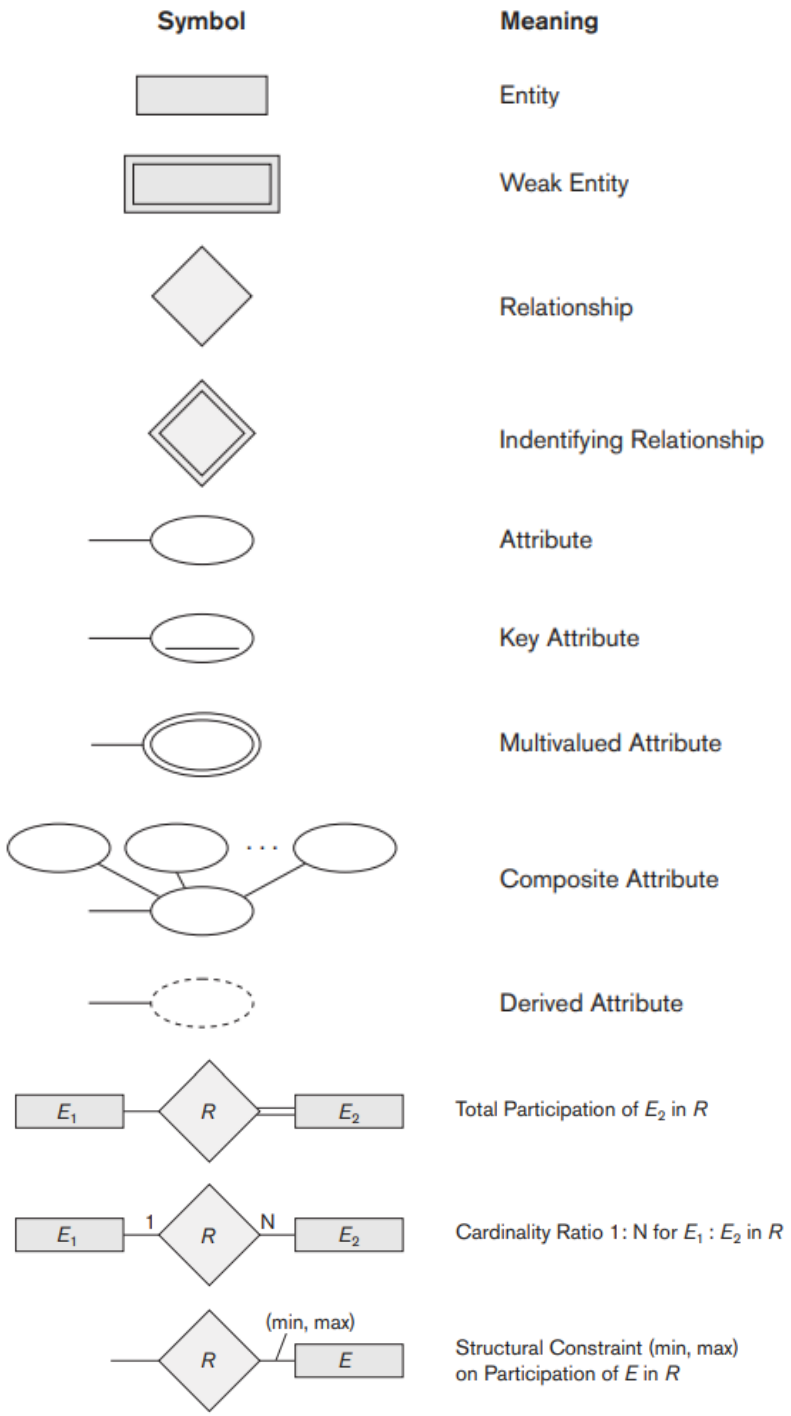
- **Attribute vs. Relationship:** If an attribute actually references another entity (e.g., "Department" in Student), it should be promoted to a relationship.

- **Attribute vs. Entity:** If an attribute has many of its own attributes (e.g., "Department" has Name, Manager, Locations), it should be promoted to an Entity.
- **Demotion:** Conversely, a simple entity related to only one other entity might be demoted to an attribute.

3.7.4 Alternative (Min, Max) Notation

An alternative to the 1:N / Double-line notation is using (min, max) pairs next to the entity.

- **Format:** (min, max) where 0 <= min <= max and max >= 1.
- **Meaning:** An entity *must* participate in at least min and at most max relationship instances.
- **Interpretation:**
  - min = 0: Partial Participation.
  - min > 0: Total Participation.
- *Note:* Figure 3.15 displays the COMPANY schema using this notation.



Chapter 3.9.1: Choosing between Binary and Ternary Relationships

While binary (degree 2) relationships are the most common, some situations require connecting three or more entity types. A relationship of degree 3 is called **Ternary**.

1. Ternary Relationship Concept

- **Definition:** A relationship type R of degree n connects R to n participating entity types.
- **Example:** SUPPLY (connecting **Supplier**, **Part**, and **Project**).
  - **Meaning:** An instance (s, j, p) means "Supplier s is *currently* supplying Part p to Project j".

2. Ternary vs. Three Binary Relationships

A common design mistake is assuming that three binary relationships are equivalent to one ternary relationship. They often represent **different information**.

A. The Difference in Meaning

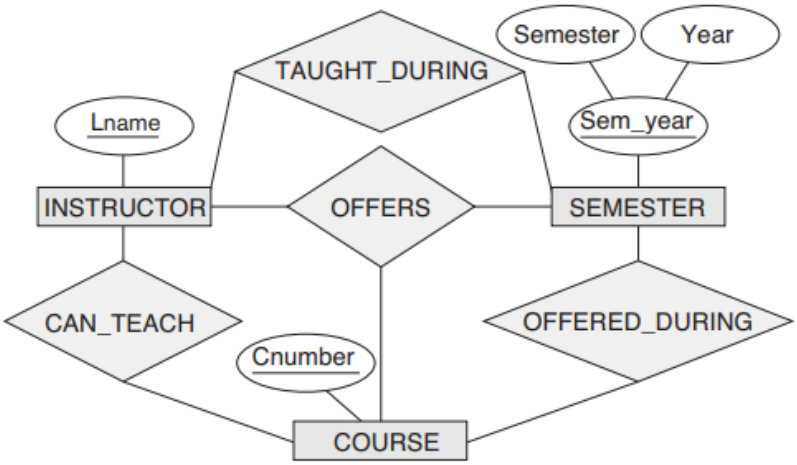
Consider the SUPPLY example again vs. three separate binaries:

1. CAN_SUPPLY (Supplier <-> Part): Supplier s *can* supply Part p (to anyone).
2. USES (Project <-> Part): Project j *uses* Part p.
3. SUPPLIES (Supplier <-> Project): Supplier s supplies *something* to Project j.

- **The Logic Gap:** Just because Supplier A *can* make Part X, Project B *uses* Part X, and Supplier A *supplies* Project B, it **does not mean** Supplier A is supplying Part X to Project B. (They might be supplying Part Y, while someone else supplies Part X).
- **Conclusion:** If the specific combination (s, j, p) is unique and significant, a ternary relationship is required.

B. Another Example: Course Offers

Consider OFFERS (Instructor, Course, Semester).



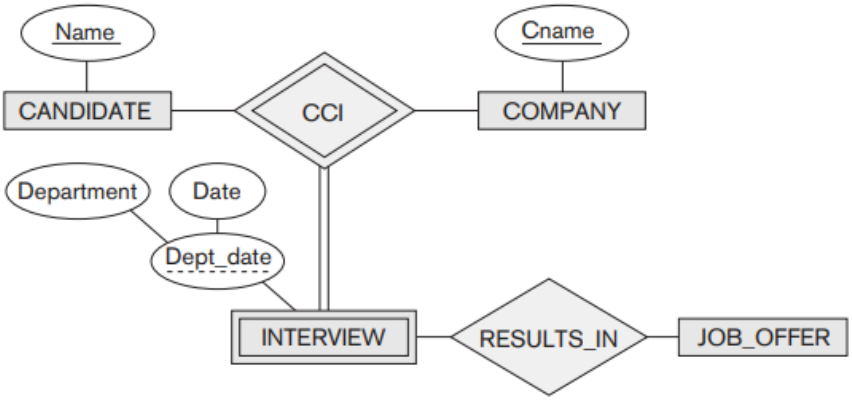
- **Ternary OFFERS:** Instructor i offers Course c during Semester s.
- **Binary Relationships:**
  - CAN_TEACH (Instructor <-> Course)
  - TAUGHT_DURING (Instructor <-> Semester)
  - OFFERED_DURING (Course <-> Semester)
- **Redundancy:** In this specific scenario, TAUGHT_DURING and OFFERED_DURING can be *inferred* from the ternary OFFERS, making them redundant. However, CAN_TEACH cannot be inferred (an instructor might be able to teach a course they aren't currently offering).

3. Representing Ternary Relationships in Binary-Only Tools

Some design tools only support binary relationships. In these cases, a ternary relationship must be converted using a **Weak Entity Type**.

- **Method:**
  1. Create a weak entity (e.g., SUPPLY).
  2. It has **no partial key**.
  3. It has **three identifying relationships** connecting to the three owners (SUPPLIER, PART, PROJECT).
  4. An instance is identified by the combination of the three owner entities.

4. Weak Entities with Ternary Identifying Relationships

A weak entity can also be identified by a ternary relationship (having multiple owners).

- **Example:** INTERVIEW weak entity.
- **Owners:** CANDIDATE and COMPANY.
- **Partial Key:** Dept_date (Department + Date).
- **Meaning:** An interview is uniquely identified by *who* is interviewing, at *which* company, and the *date/dept* combination.